

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Оглавление

Функции	2
Пользовательские функции	2
Пустая функция	2
Встроенные функции	3
Параметры функций	4
Инструкция return	5
Позиционные аргументы	6
Именованные аргументы (ключевые)	7
Переменное количество параметров в функции	7
Строки документации	10
Анонимные функции	10
Точка входа в программу	13
Область видимости	14
Локальные/вложенные переменные	14
Глобальные переменные	14
Ключевое слово global	15
Ключевое слово nonlocal	16
Общие рекомендации по написанию кода (III)	17
Несколько слов о разработке	17

Функции

Пользовательские функции

Для того чтобы многократно использовать одни и те же фрагменты кода в своих программах, применяются функции.

Создание функции происходит путём объявления ключевого слова **def** (англ. definition «определение»), далее **имени функции** и **круглых скобок**, после которых идёт символ **двоеточия**. Как и при работе с условиями и циклами тело функции должно содержать отступ в 4 пробела.

```
In [1]: def my_first_function():  
...:     print('This is my_first_function text')
```

Таким образом, мы создали простейшую функцию с именем **my_first_function**, при вызове которой на экране отобразится текст “This is my_first_function text”. Имя функции должно быть в змеиной нотации.

Важный момент: код функции должен быть написан до того, как он будет вызван, в ином случае интерпретатор выдаст ошибку.

Чтобы вызвать эту функцию в нужном месте программы (а без вызова она не сработает) достаточно напечатать её имя с круглыми скобками.

```
In [2]: my_first_function()  
This is my_first_function text
```

Тогда выполнится тело функции, и в данном случае выведется текст “This is my_first_function text”.

Код функции может содержать в себе такие конструкции, как циклы, списки, условные операторы, и любые другие, применяемые вне функций. Вызывать функцию в программе мы можем нужное количество раз.

Пустая функция

Когда вы создаёте каркас своей будущей программы, то часто будете применять пустые функции. Мы не можем создать функции без тела, иначе получим ошибку, но можно добавить ключевое слово **pass**, чтобы код смог выполняться..

```
In [3]: def my_first_function():  
...:     pass  
  
In [4]: my_first_function()
```

Если вызвать такую функцию, что ничего не произойдёт. Пустые функции нужны, чтобы видеть полную картину будущей программы. Одну за одной, в процессе создания программы, вы будете заполнять нужным функционалом.

Встроенные функции

Стоит сказать, что встроенные в Python функции мы начали использовать с самого начала изучения. Например, функция `print()`. Передавая ей в качестве параметра свой текст, мы выводим его на экран. Точно так же мы можем передавать разный текст, каждый раз вызывая нашу функцию. Для этого в скобках нам необходимо обозначить имя параметра, к которому мы будем далее обращаться.

В процессе написания кода, вы будете часто использовать такие встроенные функции, как `len()`, `print()`, `sum()`, `sorted()`, `format()`, `open()` и другие.

Полный список встроенных функций приводился в одном из прошлых уроков, напомним его тут ещё раз (полужирным шрифтом выделены те из них, что мы будем использовать постоянно)

A <code>abs()</code> <code>aiter()</code> <u><code>all()</code></u> <u><code>any()</code></u> <code>anext()</code> <code>ascii()</code>	E <u><code>enumerate()</code></u> <code>eval()</code> <code>exec()</code>	L <u><code>len()</code></u> <u><code>list()</code></u> <code>locals()</code>	R <u><code>range()</code></u> <code>repr()</code> <u><code>reversed()</code></u> <u><code>round()</code></u>
B <code>bin()</code> <code>bool()</code> <code>breakpoint()</code> <code>bytearray()</code> <code>bytes()</code>	F <code>filter()</code> <u><code>float()</code></u> <code>format()</code> <code>frozenset()</code>	M <u><code>map()</code></u> <u><code>max()</code></u> <code>memoryview()</code> <u><code>min()</code></u>	S <u><code>set()</code></u> <code>setattr()</code> <u><code>slice()</code></u> <u><code>sorted()</code></u> <code>staticmethod()</code> <u><code>str()</code></u> <u><code>sum()</code></u> <code>super()</code>
C <code>callable()</code> <u><code>chr()</code></u> <code>classmethod()</code> <code>compile()</code> <code>complex()</code>	G <code>getattr()</code> <code>globals()</code>	N <code>next()</code>	T <u><code>tuple()</code></u> <u><code>type()</code></u>
D <code>delattr()</code> <u><code>dict()</code></u> <u><code>dir()</code></u> <code>divmod()</code>	H <code>hasattr()</code> <code>hash()</code> <u><code>help()</code></u> <code>hex()</code>	O <code>object()</code> <code>oct()</code> <u><code>open()</code></u> <u><code>ord()</code></u>	V <code>vars()</code>
	I <u><code>id()</code></u> <u><code>input()</code></u> <u><code>int()</code></u> <u><code>isinstance()</code></u> <code>issubclass()</code> <code>iter()</code>	P <code>pow()</code> <u><code>print()</code></u> <code>property()</code>	Z <u><code>zip()</code></u>
			<code>__import__()</code>

Параметры функций

Параметр – это переменная в определении функции, указывающая аргумент, который функция должна принять.

```
In [1]: def my_first_function(target_text):  
...:     print(target_text)
```

В коде выше мы создали у функции `my_first_function` один параметр, под названием `target_text`. Теперь попробуем вызвать её с разными параметрами:

```
In [2]: my_first_function('this is new text ')  
this is new text  
  
In [3]: my_first_function('Another text')  
Another text
```

По сути своей мы создали аналог встроенной функции `print`, который выводит на экран тот текст, который ему передан. В качестве параметра мы можем передавать любые значения и переменные и использовать их уже в теле функции.

```
In [4]: def sum_digits_func(var_1, var_2):  
...:     print(var_1 + var_2)  
...:  
  
In [5]: int_1, int_2, = 5, 12  
  
In [6]: sum_digits_func(int_1, int_2)  
17
```

В примере выше функция получает на вход два параметра, далее код функции суммирует их и выводит результат на экран.

Аналогичный результат мы можем получить без использования имён переменных, передавая в вызов функции сразу нужные значения:

```
In [7]: sum_digits_func(456, -300)  
156
```


Инструкция return

Мы так же можем вернуть результат работы функции в какую-то переменную. Для этого нам необходимо «объяснить» компилятору, что делать по завершению работы этой функции ключевым словом `return`.

```
In [1]: def digits_func(int_1, int_2):
...:     res = (int_1 + int_2) * 2
...:
...:     return res
...:
...:
...: result = digits_func(3, 12)
...: print(result)
...:
30
```

В коде выше результат работы функции был присвоен переменной `result`, который мы и распечатали.

Функции ВСЕГДА возвращают значение, и если мы не указали это явно с помощью инструкции `return`, то функция вернёт значение по умолчанию – **None**. Мы можем распечатать, и посмотреть, что для `None` присвоен специальный тип данных `NoneType`.

```
In [2]: print_result = print('function')
function

In [3]: print(print_result, type(print_result))
None <class 'NoneType'>
```

Встроенная функция `print` выводит на экран переданный ей объект, но ничего не возвращает. Код выше как раз это и демонстрирует.

Важно понимать, что инструкция `return` прекращает выполнение кода функции, возвращая нас в то место, где эта функция была вызвана. Иногда такое поведение можно использовать для прерывания каких-то действий внутри тела функции, например таким образом мы можем выйти из цикла:

```
In [4]: def digits_func(int_1, int_2):
...:     for var in range(int_1, int_2):
...:         res = int_1 + var
...:         print(res)
...:
...:         if res > 10:
...:             return False
...:
...: digits_func(2, 7)
4
5
6
7
8
```

В этом коде мы вышли из цикла в тот момент, когда сумма чисел стала больше 10. Аналогично сработала бы и инструкция `break`, но, если бы под циклом был бы какой-то код, в случае с `break`, он бы выполнялся, в то время как в случае с `return` – нет.

Стоит отметить, что в случае завершения работы функции через оператор `return`, стоит добавлять какое-то булево значение – `True` или `False`. `True` имеет смысл добавлять, когда мы завершаем выполнение функции штатно – без ошибок, а `False` – когда произошёл нештатный выход.

```
In [5]: def check_user_input():
...:     if user_answer := int(input("Введите число от 1 до 10: ")):
...:         if 1 <= user_answer <= 10:
...:             print("Квадрат числа", user_answer, "равен", user_answer**2)
...:         else:
...:             print("Ошибка! Вы ввели число не из диапазона 1-10!")
...:             return False
...:
...:     check_user_input()
...:     check_user_input()
Введите число от 1 до 10: 5
Квадрат числа 5 равен 25
Введите число от 1 до 10: 123
Ошибка! Вы ввели число не из диапазона 1-10!
Out[5]: False
```

Таким образом через объект `False` мы показываем в коде, что что-то пошло не так.

Позиционные аргументы

Функции, которые вы видели выше, имели позиционные аргументы, то есть значения в такие аргументы подставлялись по порядку – согласно позиции имён аргументов в определении функции. Переданные в функцию параметры считываются последовательно, а их количество должно совпадать с указанным в описании функции.

```
In [1]: def digits_func(a, b):
...:     function_result = a ** b
...:     print(function_result)
...:
...:     digits_func(5, 2)
25
```

Если поменять порядок параметров, то получится совершенно другой результат

```
In [2]: digits_func(2, 5)
32
```

Именованные аргументы (ключевые)

Нередко случается, что указать порядок параметров нужно произвольно. Для таких случаев используется передача параметров по имени.

```
In [3]: def digits_func(a, b):  
...:     function_result = a ** b  
...:     print(function_result)  
...:  
...: digits_func(b=2, a=5)  
25
```

Иногда бывает необходимо указывать значение параметра по умолчанию. В этом случае мы можем передать нужные нам данные в параметр, но, если их не будет, то подставится тот, что указан изначально

```
In [4]: def mult_digits_func(a, b=10):  
...:     function_result = a * b  
...:     return function_result  
...:  
...: result_1 = mult_digits_func(5)  
...: result_2 = mult_digits_func(5, 20)  
...: print(result_1)  
...: print(result_2)  
50  
100
```

Переменное количество параметров в функции

В Python можно передать переменное (неизвестное) количество параметров двумя способами:

- `*args` для именованных (позиционных) аргументов
- `**kwargs` для именованных (ключевых - key) аргументов

Аргументы функции в Python, обозначаемые `*` и `**` на самом деле это **коллекции** - списки, кортежи или словари. Первые два передают позиционные аргументы, словари - именованные аргументы. Имена параметров `args` и `kwargs` могут быть любыми, но общепринятыми являются именно эти.

Чтобы передать функции несколько аргументов в виде коллекции, коллекцию надо распаковать. Это делается одной или двумя звездочками. Не обязательно названия должны быть `*args` и `**kwargs`, они могут быть любыми, главное здесь – оператор распаковки – звёздочка, мы с ним уже встречались в третьем уроке про типы данных.

Ниже пример такой распаковки:

```
In [5]: def check_args(*params):
...:     if len(params) == 0:
...:         print("Parameters wasn't found!")
...:         return False
...:
...:     params_qnt = 0
...:     params_sum = 0
...:     for elem in params:
...:         params_qnt += 1
...:         params_sum += elem
...:
...:     print("Params quantity:", params_qnt)
...:     print("Params sum value:", params_sum)
...:
...: check_args()
...: check_args(10, 15, 5)
Parameters wasn't found!
Params quantity: 3
Params sum value: 30
```

При вызове функции все позиционные параметры заносятся в кортеж `params`, работа с которым проходит по стандартным правилам. Далее в коде мы считаем в переменную `params_qnt` количество переданных параметров, а в переменной `params_sum` получаем сумму этих параметров.

Распаковку можно выполнить прямо в вызове функции.

```
In [6]: def print_args(*args):
...:     print(args)
...:     print(len(args))
...:     print(type(args))
...:
...: my_users = ('Bill', 'Bob', 'Alice')
...: print_args('Admin', *my_users)
('Admin', 'Bill', 'Bob', 'Alice')
4
<class 'tuple'>
```

Здесь мы распаковываем кортеж, который далее идёт в качестве аргументов функции. Обратите внимание, что длина `*args` будет содержать в себе все переданные аргументы.

А ****kwargs** у нас принимает именованные аргументы и выводит словарь.

```
In [7]: def func(**kwargs):
...:     return kwargs
...:
...:     print("Login:Password\n")
...:     print(func(Admin="Administrator", Zlodey="black_hacker", Luser="Losharik"))
Login:Password
{'Admin': 'Administrator', 'Zlodey': 'black_hacker', 'Luser': 'Losharik'}
```

Также, существует возможность комбинировать позиционные и именованные аргументы. Если в функцию необходимо передать параметры одного и другого типа, то всего лишь их нужно указать через запятую. Но при этом позиционные аргументы передаются ДО ключевых.

```
In [8]: def func(*args, **kwargs):
...:     print(''.join(args) + '\n')
...:     return kwargs
...:
...:     target_dict = dict(Admin="Administrator", Zlodey="black_hacker", Luser="Losharik")
...:     print(func('Login:', 'Password', **target_dict))
Login:Password
{'Admin': 'Administrator', 'Zlodey': 'black_hacker', 'Luser': 'Losharik'}
```

Строки документации

Python имеет одну особенность, называемую **строками документации**, обычно обозначаемую сокращённо **docstrings**. Этот инструмент помогает лучше документировать программу, и облегчает её понимание. Строку документации, в отличие от комментариев, можно получить из функции, даже во время выполнения программы.

Доступ к строке документации функции можно получить с помощью атрибута этой функции `__doc__`, который нужно записать через точку, и вывести с помощью встроенной функции `print`. В примере ниже это будет `print(number_max.__doc__)`

Практически аналогичный вывод можно получить, используя функцию `help()` `help(number_max)`

```
In [1]: def get_max_int(x, y):
...:     """
...:     Функция приводит переданные аргументы к числовому виду
...:     И находит большее число
...:     """
...:     x, y = int(x), int(y)
...:
...:     if x > y:
...:         print(x, "наибольшее")
...:     else:
...:         print(y, "наибольшее")
...:
...: get_max_int(99, 89)
...: print(get_max_int.__doc__)
99 наибольшее

Функция приводит переданные аргументы к числовому виду
И находит большее число
```

Старайтесь не обращаться к строкам документации в коде – они являются сервисным инструментом и прежде всего предназначены для программиста, а не для пользователя.

Анонимные функции

Бывают ситуации, когда создавать полноценную функции в коде избыточно, в этих случаях мы можем обратиться к так называемым неименованным или анонимным функциям.

Создаются они с помощью инструкции **lambda**.

lambda <параметры>: тело функции

Функции `lambda` могут содержать только одну строку кода. Также `lambda` функции, в отличие от обычной, не требуется инструкция `return`.

```
In [1]: func = lambda x, y: x + y
...: print(func(10, 25))
35
```

В этом примере мы создали анонимную функцию и поместили её в переменную `func`. То есть дальше в коде, чтобы использовать эту функцию, мы должны обращаться к ней по этому имени – что мы и сделали во второй строке: передали в `func` два аргумента, получили их сумму и распечатали результат.

Реализация этого же кода с помощью именованной функции будет выглядеть гораздо объёмнее. Сравните:

```
In [2]: def func(x, y):
...:     return x + y
...:
...: print(func(10, 25))
35
```

Лямбда-функции могут быть переданы без необходимости в предварительном присваивании переменной. Её можно использовать внутри стандартной функции.

```
In [3]: def my_func(n):
...:     return lambda a: a ** n
...:
...: exponential = my_func(3)
...: print(exponential(8))
...:
512
```

В теле лямбда функции можно создавать условия тернарным оператором

```
In [4]: check_num = lambda x: 'Число чётное\n' if x % 2 == 0 else 'Число не чётное\n'
...: print(check_num(21))
...: print(check_num(18))
Число не чётное
Число чётное
```

Лямбда-функции прекрасно подходят для сортировки, где они довольно часто применяются.

Допустим, у нас есть словарь результатов голосования за лучшую книгу:

```
In [1]: results = {'Мастер и Маргарита': 17439, 'Собачье сердце': 17016,
...:               'Двенадцать стульев': 16903, 'Мёртвые души': 16345,
...:               'Граф Монте-Кристо': 15147}
```

Как нам вывести отсортированный список, используя функцию `sorted`? Мы знаем, что у словаря есть метод `items`, но результат сортировки будет по буквам ключей:

```
In [2]: print(*sorted(results.items()), sep='\n')
('Граф Монте-Кристо', 15147)
('Двенадцать стульев', 16903)
('Мастер и Маргарита', 17439)
('Мёртвые души', 16345)
('Собачье сердце', 17016)
```

А как отсортировать словарь по значениям, но получить и ключи, и значения одновременно? Есть очень удобный способ!

Вторым аргументом функции `sorted` мы передаём ключ сортировки – аргумент `key`. Встроенного ключа для получения значений из пары ключ и значения нет. Поэтому придётся нам написать его самому.

Если сделать это через обычную функцию, то код будет выглядеть так. Метод `.items()` возвращает нам кортеж (ключ, значение), значит функция должна принимать этот кортеж, а возвращать должна только второй элемент – значение:

```
In [3]: results = {'Мастер и Маргарита': 17439, 'Собачье сердце': 17016,
...:               'Двенадцать стульев': 16903, 'Мёртвые души': 16345,
...:               'Граф Монте-Кристо': 15147}
...:
...: def sorted_alg(args):
...:     return args[1]
...:
...: print(*sorted(results.items(), key=sorted_alg), sep='\n')
('Граф Монте-Кристо', 15147)
('Мёртвые души', 16345)
('Двенадцать стульев', 16903)
('Собачье сердце', 17016)
('Мастер и Маргарита', 17439)
```

Эта функция выполняет только одно действие – возвращает второй элемент кортежа, а код её слишком большой для именованной функции – вот тут и пригодится анонимная функция:

```
In [4]: results = {'Мастер и Маргарита': 17439, 'Собачье сердце': 17016,
...:               'Двенадцать стульев': 16903, 'Мёртвые души': 16345,
...:               'Граф Монте-Кристо': 15147}
...:
...: print(*sorted(results.items(), key=lambda args: args[1]), sep='\n')
('Граф Монте-Кристо', 15147)
('Мёртвые души', 16345)
('Двенадцать стульев', 16903)
('Собачье сердце', 17016)
('Мастер и Маргарита', 17439)
```

Как видите, мы смогли сэкономить 2 строки кода, не потеряв в читабельности!

Точка входа в программу

При запуске любой программы на языке python кроме переменных, создаваемых в коде, есть ещё и сервисные переменные. Чтобы увидеть их, можно воспользоваться функцией `dir()`. Все они начинаются и заканчиваются двумя нижним подчёркиваниями, что говорит об их скрытости.

Среди этих объектов есть поле `__name__`, которое содержит имя того объекта, который вызвал нашу программу. Если мы запустили файл через интерпретатор python, то это имя будет выглядеть как строка `'__main__'`.

Если же мы импортировали файл этот файл в другой код, то там будет имя этого файла (о модулях и импортах мы поговорим в следующем уроке).

Таким образом, для проверки источника запуска программы используется следующее условие:

```
if __name__ == '__main__':
    # Код программы
```

Это условие называется «точкой входа в программу». Такая конструкция служит двум важным целям:

- Мы явно показываем место запуска нашего кода. Человек, читающий наш код, сразу увидит то место, откуда начинается выполнение программы.
- Код программы внутри условия будет выполняться только в том случае, если программа вызвана, как файл. Если же мы импортировали программу, то код внутри точки входа не сработает – это позволяет тестировать программу-модуль, не нарушая её работоспособность в других файлах

Конструкция `if __name__ == '__main__':` позволяет логически организовать код, и сделать его удобочитаемым. Схематично это можно представить так:

```
In [1]: def foo():
...:     print('Hello foo!')
...:
...: def bar():
...:     print('Hello bar!')
...:
...: def foo_bar():
...:     print('Hello foo_bar!')
...:
...: if __name__ == '__main__':
...:     foo()
...:     bar()
...:     foo_bar()
```

В домашнем задании наличие точки входа в программах больше 5-10 строк обязательно. Старайтесь сразу приучать себя использовать эту конструкцию в коде.

Область видимости

В Python переменная доступна только в той области, внутри которой она создана. Эта область называется областью видимости. Всего существует 4 области видимости, запомнить их можно с помощью аббревиатуры **LEGB**:

- **L** – Local, локальная область видимости.
- **E** – Enclosing, области видимости функций
- **G** – Global, глобальная область видимости
- **B** – Built-in, область видимости встроенных функций

Такая особенность позволяет ограничивать доступ к определенным значениям во избежание конфликтов между одинаковыми идентификаторами

Локальные/вложенные переменные

Для создания переменных, обладающих локальной и вложенной областью видимости, необходимо всего лишь поместить их в отдельный блок кода, изолированный от остальной программы. Это может быть, например, функция

```
In [1]: def my_func():
...:     input_var = 'user_string'
...:     print(input_var)
...:
...: my_func()
...: # Обратиться к переменной input_var вне функции не получится,
...: # произойдет ошибка
...: print(input_var)
user_string
-----
NameError                                Traceback (most recent call last)
Cell In[1], line 8
      5 my_func()
      6 # Обратиться к переменной input_var вне функции не получится,
      7 # произойдет ошибка
----> 8 print(input_var)

NameError: name 'input_var' is not defined
```

Как видите, код программы выполнен только отчасти – python создал функцию вместе с её переменными, вызвал её, а дальше на 8 строке кода мы попытались распечатать эту вложенную переменную, но её нет в глобальной области видимости, где и находится весь код программы.

Глобальные переменные

```
In [2]: def my_func():
...:     print(input_var)
...:
...: input_var = 'user_string'
...: my_func()
...:
...: print(input_var)
user_string
user_string
```

Чтобы иметь возможность обратиться к переменной в любой части программы, её следует объявить в глобальной области видимости – то есть вне функций.

Как видите, в этот раз мы объявили переменную в глобальной области и смогли получить к ней доступ как внутри функции, так и вне её.

Если же объявлены несколько переменных с одинаковым именем, то python будет искать их по порядку LEGB – переменные “a” никак между собой не пересеклись и имеют разные значения, так находятся в разных областях видимости.

```
In [3]: a = 3**5    # глобальная переменная
...:
...:
...: def foo():
...:     # локальная переменная
...:     a = 3 + 23
...:     print(a)
...:
...:
...: foo()          # 26
...: print(a)
26
243
```

Конечно, пользоваться одинаковыми именами переменных в реальной программе не следует.

Ключевое слово global

Использование ключевого слова global необходимо в тех случаях, когда есть потребность сделать локальную переменную глобальной.

```
In [4]: def func():
...:     global var
...:     var = 2**5 + 8
...:
...: func()
...: print(var)
40
```

Старайтесь как можно реже пользоваться этой возможностью, поскольку изменение глобальных переменных в локальных областях видимости неочевидно и может привести к логическим ошибкам в коде.

```
In [5]: def func():
...:     global var
...:     var = 2**5 + 8
...:
...: var = 5
...: func()
...: print(var)
40
```

В примере слева мы объявили глобальную переменную, а потом вызвали функцию, в которой изменила её значение, но, когда в коде несколько десятков или даже сотен строк, узнать, где это произошло очень сложно, поэтому разумнее использовать для изменения не оператор global, а аргументы и другие подходы.

Ключевое слово nonlocal

Нелокальные области видимости встречаются, когда вы определяете функции внутри функций. При объявлении переменной через `nonlocal` она будет ссылаться на переменную с таким же именем в ближайшем замыкании, исключая глобальные переменные.

```
In [1]: x = 0
...:
...:
...: def outer():
...:     x = 1
...:
...:     def inner():
...:         nonlocal x
...:         x = 20
...:         print("inner:", x)
...:
...:     inner()
...:     print("outer:", x)
...:
...: outer()
...: print("global", x)
...:
inner: 20
outer: 20
global 0
```

Если мы уберём `nonlocal` из этого примера, то получим другой вывод, так как область видимости изменится на локальную.

```
In [2]: x = 0
...:
...:
...: def outer():
...:     x = 1
...:
...:     def inner():
...:         x = 20
...:         print("inner:", x)
...:
...:     inner()
...:     print("outer:", x)
...:
...: outer()
...: print("global", x)
...:
inner: 20
outer: 1
global 0
```

Общие рекомендации по написанию кода (III)

1. Соблюдайте порядок в коде. Не нужно объявлять переменные до функций. В редких случаях такой синтаксис допускается, но сначала всегда создаём функции, потом переменные, потом работаем и с теми, и с другими
 - а. Так же не объявляйте переменные между функциями, блок с функциями отдельно, блок с переменными отдельно.
2. Не нужно вкладывать функцию в функцию – подобный синтаксис является рабочим, но использовать его не стоит. Мы будем о нём говорить в уроке про декораторы, но пока создавайте только один уровень вложенности для функций.
3. Активнее используйте аргументы функции и оператор `return` – подобное поведение понятно и легко читаемо.

Несколько слов о разработке

Перед выполнением какого-то задания или написанием собственного кода нужно продумать его структуру. Для этого можно нарисовать блок-схему на листе бумаги, в том же `paint`е или специализированные инструменты вроде <https://programforyou.ru/block-diagram-redactor>

Подобный подход позволит вам учесть многие факторы ещё на этапе разработки логики, а не в момент её написания.

Кроме текста, которым вы можете дополнить такую блок-схему, нужно обращать внимание на типы фигур. Обычно начало/конец программы описывается овалом с текстом Начало/Конец, логические блоки оформляются прямоугольниками, для условий используется горизонтальный ромб, а для цикла – шестиугольник

Внутри самих фигур вы можете описать выполняемую этой фигурой задачу - создание переменной, проверку значения, печать данных

Ниже будет пример блок-схемы из прошлого урока, где мы перебирали пословицы и считали сумму чисел.

Попробуйте перед выполнением задания 5 нарисовать такую же блок-схему.

